PREMIER UNIVERSITY, CHATTOGRAM

Department of Computer Science & Engineering



Project Report

on

"Predictive Analytics of Steam Market Trends and Game Success."

Course Name: Machine Learning Laboratory.

Course code: CSE - 4312.

SUBMITTED BY

Name: Nishat Afroz

ID: 0222220005101145

Name: Mohammed Walid

ID: 0222220005101147

Supervised By

Tanni Dhoom

Assistant Professor

Department of Computer Science & Engineering Premier University, Chattogram

25th May, 2026

Table of Contents

1. Introduction....1
2. Related Works....1
3. Objectives....2
4. Problem Definition and Algorithm....2
    4.1 Problem Definition....2
    4.2 Algorithm....4
5. Methodology....14
    5.1 Dataset Description....14
    5.2 Hypothesis....14
    5.3 Web Scraping....15
    5.4 Data Preprocessing and Cleaning....16
    5.5 Machine Learning Models Used....20
    5.6 Independent and Dependent Variables....22
    5.7 Training and Testing Method....23
    5.8 Cross-Validation....24
    5.9 Hyperparameter Tuning....25
    5.10 Evaluation Criteria....26
    5.11 Performance Analysis....28
6. Result and Analysis....29
    6.1 Game Success Prediction....29
    6.2 Popular vs Non-popular Games....31
    6.3 High Discount Prediction....32
    6.4 Game Recommendation System....34
    6.5 Game Genre Classification....35
7. Discussion....35
8. Future Work....36
9. Conclusion....36
10. Reference....37

1. Introduction

The rapid growth of the gaming industry and digital distribution platforms such as Steam has generated a massive amount of game-related data, including user ratings, prices, genres, reviews, release information and popularity metrics. Analyzing this data using Machine Learning techniques can help identify patterns that influence a game's success and improve recommendation systems for users. Predicting whether a game will become popular, estimating player preferences and recommending similar games are important tasks for both game developers and online gaming platforms.

Traditional Machine Learning algorithms such as Support Vector Machine (SVM), Logistic Regression, Decision Tree, Random Forest and XGBoost were used to classify games into high-popular and low-popular categories. In addition, a recommendation system was developed to suggest similar games based on game features and tags. The project also explores Natural Language Processing techniques by comparing a traditional TF-IDF plus Logistic Regression model with a Transformer-based DistilBERT model for text-based analysis. The project is important because recommendation and popularity prediction systems are widely used in modern gaming platforms to improve user experience, increase engagement and support better business decisions.

Experimental results show that ensemble methods such as Random Forest and XGBoost achieved strong prediction performance, while the DistilBERT model outperformed traditional Bag-of-Words approaches in text classification tasks. Feature importance analysis revealed that factors like current price, discount percentage, release year, Steam achievements and multiplayer-related features significantly affect game popularity. The recommendation system successfully generated relevant game suggestions based on content similarity.

2. Related Works

Several studies have explored the use of Machine Learning and recommendation systems for analyzing video game popularity and improving game recommendations on platforms such as Steam. These works mainly focus on popularity prediction, recommendation algorithms and user behavior analysis. The following section discusses important previous works, their methods, limitations and how the proposed project differs from and improves upon them.

A study by Andraž De Luisa et al.[1] focused on predicting the popularity of Steam games using Bayesian Machine Learning models. The authors analyzed features such as price, release date, genres, supported languages and game size to estimate player counts and game popularity. Their approach mainly used hierarchical Bayesian methods to understand the relationship between game characteristics and popularity.

Our approach is more comprehensive because it combines both traditional Machine Learning and Deep Learning techniques within one system. Ensemble methods such as Random Forest and XGBoost achieved strong accuracy while DistilBERT improved text understanding compared to traditional approaches. The inclusion of recommendation functionality also makes the system more practical for real-world gaming platforms.

Hu and Wang (2024)[2] analyzed Steam game popularity using several Machine Learning algorithms including Logistic Regression, SVM, Decision Tree, XGBoost, LightGBM, Deep Neural Networks and CNN models. Their research compared multiple algorithms to identify which performed best for popularity prediction tasks.

Our project provides a broader gaming analytics framework instead of focusing on a single prediction task. The comparison between TF-IDF and DistilBERT also demonstrates the advantages of transformer models in handling textual game data more effectively than traditional methods.

Liangwei Yang et al.[3] proposed a recommendation framework called SCGRec, which uses Graph Neural Networks (GNNs) and social-aware contextual modeling to recommend personalized games to users on Steam. The study focused on user interactions, social relationships, and gameplay behavior to improve recommendation quality.

Their work mainly relies on complex graph neural networks and user interaction graphs, which require large-scale user activity data. In comparison, our recommendation system uses content-based filtering with game metadata, tags, genres and similarity measures.

3. Objectives

The project mainly focuses on the following objectives:

1. Predict whether a game belongs to the high-popular or low-popular category.
2. Analyze which features influence game popularity the most.
3. Build a recommendation system to suggest similar games.
4. Compare traditional NLP techniques with transformer-based Deep Learning models.
5. Evaluate the effectiveness of different Machine Learning algorithms.

4. Problem Definition and Algorithm

4.1 Problem Definition

This project addresses multiple machine learning tasks on a video game dataset, aiming to extract meaningful insights and build predictive systems. The primary problems tackled include:

1. **Game Success Prediction (Regression Task)**
   The objective is to predict a continuous success score for a game based on features such as reviews, metascore, price, tags, platform and release information.
   - **Input:** Game features (e.g., price, tags, platform, release year, developer, publisher)
   - **Output:** A numerical success score representing the overall performance of a game

2. **Game Popularity Classification (Binary Classification Task)**
   The goal is to classify games as popular or not popular based on review count.
   - **Input:** Numerical features such as price, metascore, discount, and release year
   - **Output:** Binary label (1 = popular, 0 = not popular)

3. **High Discount Prediction (Binary Classification Task)**
   This task predicts whether a game will have a high discount (>50%).
   - **Input:** Game features including metascore, reviews, price and tags
   - **Output:** Binary label (1 = high discount, 0 = low/no discount)

4. **Game Recommendation System (Similarity-Based Task)**
   The objective is to recommend similar games based on content.
   - **Input:** Game title along with textual features (tags and summary)
   - **Output:** List of top-N similar games

5. **Game Genre Classification (Multi-class Classification Task)**
   This task aims to predict the genre of a game from its textual description.
   - **Input:** Game summary (text data)
   - **Output:** Genre label (e.g., Action, Adventure, Strategy, etc.)

Importance of the Problem

These problems are highly relevant in the modern gaming industry and data-driven platforms for several reasons:

- **Improved User Experience:**
  Recommendation systems help players discover new games tailored to their interests.
- **Business Decision Making:**
  Predicting game success and popularity enables developers and publishers to make informed decisions regarding pricing, marketing, and release strategies.
- **Revenue Optimization:**
  Understanding discount patterns helps optimize pricing strategies and maximize sales.

- **Automation of Content Understanding:**
  Genre classification using NLP reduces manual labeling and enables scalable content organization.
- **Industry Relevance:**
  These tasks reflect real-world applications used by platforms like Steam, Netflix, and Amazon, making the problem both practical and impactful.

4.2 Algorithm

4.2.1 Pseudocode for Game Success Prediction using Random Forest and XGBoost

```
BEGIN

1. LOAD dataset from CSV file

2. DATA PREPROCESSING
  - Handle missing values
  - Convert date columns to datetime format

3. CREATE TARGET VARIABLE (SUCCESS SCORE)

  a. Compute:
    log_reviews = log(1 + review_count)

  b. Normalize metascore:
    metascore_norm = metascore / 100

  c. Create success score:
    success = log(1 + (log_reviews + metascore/10))

4. FEATURE ENGINEERING

  a. TAG FEATURES
    - Split tags into list
    - Apply MultiLabelBinarizer
    - Keep top N most frequent tags

  b. PLATFORM FEATURES
    - Create binary columns:
      windows, mac, linux

  c. RELEASE FEATURES
    - Extract release_year and release_month
    - Compute game_age = current_year - release_year
```

d. PRICE FEATURES
- current_price
- discount_percent
- price_after_discount
- price_per_year

e. DEVELOPER & PUBLISHER FEATURES
- Select top N developers and publishers
- Replace others with "Other"
- Apply one-hot encoding

5. CREATE FINAL FEATURE MATRIX
- Combine:
  tags + price features + platform + release + developer/publisher
- Fill missing values with 0

6. DEFINE:
X = feature matrix
y = success score

7. SPLIT DATA
- Training set: 80%
- Testing set: 20%

---
8. MODEL 1: RANDOM FOREST REGRESSION

a. Initialize Random Forest model

b. Define hyperparameter search space:
n_estimators
max_depth
min_samples_split
min_samples_leaf
max_features

c. Perform Randomized Search with cross-validation

d. Train model on training data

e. Predict on test data

f. Evaluate using:
Mean Squared Error (MSE)
Mean Absolute Error (MAE)
R^2 Score

g. Compute feature importance

---
9. MODEL 2: XGBOOST REGRESSION

a. Initialize XGBoost model

b. Define hyperparameter search space:
n_estimators
max_depth
learning_rate
subsample
colsample_bytree
gamma
regularization terms

c. Perform Randomized Search with cross-validation

d. Train model on training data

e. Predict on test data

f. Evaluate using:
MSE, MAE, R^2 Score

g. Compute feature importance

---
10. MODEL COMPARISON

- Compare Random Forest and XGBoost using:
R^2 Score
MSE
MAE

- Identify best performing model

---
11. OUTPUT RESULTS

- Print evaluation metrics for both models
- Display top important features
- Report best model

END

4.2.2 Comparison of Machine Learning Models for Game Popularity Classification

```
##  Popular vs Non-Popular Games
BEGIN
1. LOAD dataset from CSV file

2. CHECK for missing values

3. CREATE target variable (popular):
    COMPUTE median of review_count
    FOR each row:
        IF review_count > median:
            popular = 1
        ELSE:
            popular = 0

4. SELECT features:
    original_price
    current_price
    metascore
    discount_percent
    release_year

5. REMOVE rows with missing values in selected features

6. DEFINE:
    X = selected features
    y = target (popular)

7. SPLIT data into training and testing sets:
    80% training
    20% testing

8. SCALE features (for SVM and Logistic Regression):
    FIT scaler on training data
    TRANSFORM training data
    TRANSFORM testing data

9. TRAIN MODELS:

  a) SVM Model:
      USE RBF kernel
      TRAIN on scaled training data
      PREDICT on scaled test data
```

```
  b) Logistic Regression Model:
       TRAIN on scaled training data
       PREDICT on scaled test data

  c) Random Forest Model:
       TRAIN on original (non-scaled) training data
       PREDICT on test data

10. EVALUATE MODELS:

    FOR each model:
        CALCULATE accuracy
        GENERATE confusion matrix
        GENERATE classification report

11. COMPARE MODELS:
    STORE accuracy of each model

12. FIND best model:
    SELECT model with highest accuracy

13. PRINT results:
    accuracies of all models
    best performing model

END
```

4.2.3 Pseudocode for High Discount Prediction using Logistic Regression and Decision Tree

```
BEGIN

1. LOAD dataset from CSV file

2. DATA PREPROCESSING

  a. HANDLE TAGS
    - Split 'tags' column into list format
    - Apply MultiLabelBinarizer to convert tags into binary features
    - Append encoded tag features to dataset
    - Remove original 'tags' column

  b. SELECT TOP TAGS
    - Identify top 30 most frequent tags
    - Keep only these tag columns
```

3. CREATE TARGET VARIABLE

- IF discount_percent > 50:
    high_discount = 1
  ELSE:
    high_discount = 0

4. SELECT FEATURES

- metascore
- review_count
- release_year
- original_price
- top tag features

5. HANDLE SKEWED DATA

- Apply log transformation:
    log_price = log(1 + original_price)
    log_reviews = log(1 + review_count)

- Remove original_price and review_count

6. VISUALIZATION (optional)

- Plot distributions before and after log transformation

7. SPLIT DATA

- Training set: 80%
- Testing set: 20%
- Use stratified split based on target variable

8. HANDLE CLASS IMBALANCE

- Apply SMOTE on training data only
- Generate balanced training dataset

9. FEATURE SCALING

- Fit StandardScaler on training data
- Transform both training and testing data

---

10. MODEL 1: LOGISTIC REGRESSION

```
   - Train model on resampled training data
   - Predict on test data
   - Evaluate using classification report

-------------------------------------------------
11. MODEL 2: DECISION TREE

   - Initialize Decision Tree with max_depth = 5
   - Train model on resampled training data
   - Predict on test data
   - Evaluate using classification report

-------------------------------------------------
12. FEATURE IMPORTANCE ANALYSIS

   - Extract feature importance from Decision Tree
   - Rank features in descending order
   - Display top important features

-------------------------------------------------
13. OUTPUT RESULTS

   - Print classification reports for both models
   - Display feature importance table

END
```

4.2.4 Pseudocode for Game Recommendation System using TF-IDF and Cosine Similarity

```
BEGIN

1. LOAD dataset from CSV file

2. DATA PREPROCESSING

   a. CREATE content profile:
      - Combine 'tags' and 'summary' columns into a single text field
      - Replace missing values with empty string

3. TEXT VECTORIZATION (TF-IDF)

   - Initialize TF-IDF Vectorizer with English stop words
   - Fit and transform 'content_profile' into TF-IDF matrix
```

```
4. COMPUTE SIMILARITY MATRIX

  - Calculate cosine similarity between all game vectors
  - Store similarity scores in a matrix

5. DEFINE FUNCTION: get_recommendations(game_title, N)

  a. FIND index of the input game title
      IF game not found:
         RETURN "Game not found"

  b. GET similarity scores for the selected game

  c. SORT all games by similarity score (descending order)

  d. SELECT top N similar games
      - Exclude the input game itself

  e. EXTRACT indices of recommended games

  f. RETURN selected game details:
      - title
      - metascore
      - tags

6. TEST RECOMMENDATION SYSTEM

  - CALL get_recommendations("Game Name", N)
  - DISPLAY recommended games

END
```

4.2.5 Pseudocode for Game Genre Classification using Traditional NLP and Transformer Models

```
BEGIN

1. LOAD dataset from CSV file

2. CREATE TARGET VARIABLE

  - Extract primary_genre from 'tags' column
```

(take first tag as main genre)

3. DATA CLEANING

- Remove rows with missing:
  summary
  primary_genre

- Select top N (10) most frequent genres
- Filter dataset to include only these genres

4. TEXT PREPROCESSING

a. Download stopwords and lemmatizer

b. DEFINE clean_text function:
  - Convert text to lowercase
  - Remove punctuation and numbers
  - Split into tokens
  - Remove stopwords
  - Apply lemmatization
  - Join tokens back into text

c. Apply clean_text to summary column
  → create clean_summary

5. ENCODE TARGET LABELS

- Convert genre names into numeric labels using LabelEncoder

6. TRAIN-TEST SPLIT

- Split clean_summary and labels into:
  Training set (80%)
  Testing set (20%)

---

7. MODEL 1: TF-IDF + LOGISTIC REGRESSION

a. Convert text to numerical form:
  - Apply TF-IDF vectorization (max_features = 5000)

b. Train Logistic Regression model

c. Predict on test data

d. Evaluate using accuracy score

---
8. MODEL 2: DISTILBERT (TRANSFORMER)

a. Load pretrained tokenizer (DistilBERT)

b. DEFINE custom dataset class:
- Tokenize text (padding + truncation)
- Convert to tensors
- Store labels

c. Create training and testing datasets

d. Load pretrained DistilBERT model for classification
- Set number of output labels = number of genres

e. Define training parameters:
- Number of epochs
- Batch size
- Evaluation strategy

f. Initialize Trainer with:
- Model
- Training arguments
- Training dataset
- Evaluation dataset

g. Train the model

---
9. MODEL COMPARISON

- Store accuracy of:
  TF-IDF + Logistic Regression
  DistilBERT

- Create comparison table

- Plot bar graph of model accuracies

---
10. OUTPUT RESULTS

- Display accuracy scores
- Show comparison graph

- Identify better performing model

END

5. Methodology

This project applies multiple Machine Learning and Deep Learning techniques to analyze video game datasets and solve tasks related to popularity prediction, recommendation systems, discount analysis and text classification. The methodology consists of data preprocessing, feature extraction, model training, evaluation and comparative analysis of different algorithms.

5.1 Dataset Description

The dataset used in this project was collected from Steam game data and contains information related to:

- Title
- Original Price
- Current Price
- Discount
- Review Count
- Tags
- Platform
- Release Date
- Metascore
- Developer
- Publisher
- Summary

The dataset is realistic and useful because it represents actual gaming platform data used in modern recommendation and analytics systems. Since Steam contains thousands of games with diverse categories and user interactions, it provides an effective environment for Machine Learning experiments.

5.2 Hypothesis

The experiments in this project were designed based on the following hypothesis:

- Ensemble learning methods such as Random Forest and XGBoost perform better than simple linear models for popularity prediction.
- Transformer-based models such as DistilBERT achieve higher text classification accuracy than TF-IDF with Logistic Regression.

- Features such as current price, discount percentage, release year, achievements and multiplayer support significantly affect game popularity.
- Content-based recommendation systems can successfully recommend similar games using tags and metadata.

5.3 Web Scraping

The dataset used in this project was collected through a custom web scraping pipeline developed in Python. The scraping process focused on gathering video game information from the Steam platform, including pricing details, tags, reviews, genres, release information, platform support, developers, publishers and metascores.

Technologies Used

The following Python libraries were used during the web scraping process:

- **Requests** – for sending HTTP requests to Steam webpages and APIs,
- **BeautifulSoup (bs4)** – for parsing HTML content and extracting data,
- **Pandas** – for storing and organizing the collected data into DataFrames,
- **Time** – for adding delays between requests to avoid rate limiting.

Step 1: Collecting Steam App IDs

The first stage of the scraping process involved collecting Steam App IDs from Steam search pages. These App IDs uniquely identify each game on the Steam platform.

A custom function called get_app_ids() was created to:

- Visit multiple Steam search result pages,
- Parse HTML using BeautifulSoup,
- Extract App IDs from page elements,
- Store the IDs in a list.

Example:

```
for link in soup.find_all('a', {'data-ds-appid': True}):
    app_ids.append(link['data-ds-appid'])
```

Step 2: Retrieving Game Details from Steam API

After collecting App IDs, the Steam Store API was used to retrieve detailed information about each game.

A custom function named get_game_details(app_id) was created to access the Steam API endpoint:

```
api_url = f"https://store.steampowered.com/api/appdetails?appids={app_id}"
```

Example extracted fields:

```
{
   "Title": data.get("name"),
   "Current Price": final_price,
   "Discount %": discount_pct,
   "Review Count": review_count,
   "Tags": all_tags,
   "Metascore": metascore,
   "Developer": ", ".join(data.get("developers", [])),
   "Publisher": ", ".join(data.get("publishers", []))
}
```

Step 3: Creating the Dataset

After collecting all game information, the data was converted into a Pandas DataFrame:

```
df = pd.DataFrame(game_list)
```

The final dataset was then exported into CSV files for Machine Learning experiments:

```
df.to_csv("Games_dataset_full.csv", index=False)
```

5.4 Data Preprocessing and Cleaning

The raw dataset was collected from Steam game data and required several preprocessing steps to ensure consistency, accuracy and usability for analysis.

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3278 entries, 0 to 3277
Data columns (total 12 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   Title           3278 non-null   object
 1   Original Price  3278 non-null   float64
 2   Current Price   3278 non-null   float64
 3   Discount %      3278 non-null   int64
 4   Review Count    3278 non-null   int64
 5   Tags            3278 non-null   object
 6   Platform        3278 non-null   object
 7   Release Date    3275 non-null   object
 8   Metascore       1799 non-null   float64
 9   Developer       3276 non-null   object
 10  Publisher       3272 non-null   object
 11  Summary         3278 non-null   object
dtypes: float64(3), int64(2), object(7)
memory usage: 307.4+ KB
```

Figure: Original Dataset Values.

The following steps were performed:

1. Data Loading and Initial Inspection

The dataset was first loaded using pandas, and initial inspection was conducted using .head() and basic exploratory checks. Additionally, the proportion of games with no discount was computed which provided an early understanding of pricing patterns in the dataset

2. Removal of Duplicate Records

Duplicate entries were identified based on the game title, as multiple rows for the same game could distort analysis.

- Duplicates were removed.
- Only the first occurrence of each title was retained.
- A verification step confirmed that no duplicates remained.

This step ensured that each game was uniquely represented.

3. Standardization of Column Names

To improve consistency and avoid potential errors in column referencing:

- All column names were converted to lowercase.
- The leading and trailing whitespace was removed.

4. Handling Missing Values

Missing data was treated differently depending on the column type:

- **Categorical columns**:
    - developer and publisher were filled with "Unknown" to retain rows while acknowledging missing information.
- **Numerical column**:
    - metascore was filled using the median value, which is robust to skewed distributions and outliers.
- Missing values were checked before and after treatment to ensure completeness.

5. Handling Missing Dates

Rows with missing release date values were removed. Since release date is a critical variable for time-based analysis, imputing it could introduce inaccuracies.

6. Date Conversion and Feature Engineering

The release date column was converted into a datetime format. From this, two new features were extracted:

- **Release Year**
- **Release Month**

These features enabled temporal trend analysis and seasonality insights.

7. Final Null Check and Data Integrity

A final check was performed to ensure no unintended missing values remained. Any rows with invalid or unparseable dates were removed again to maintain consistency.

8. Outlier Detection

Boxplots were generated for key numerical variables:

- Original Price
- Current Price
- Review Count
- Metascore

This was done using seaborn to visually inspect potential outliers.

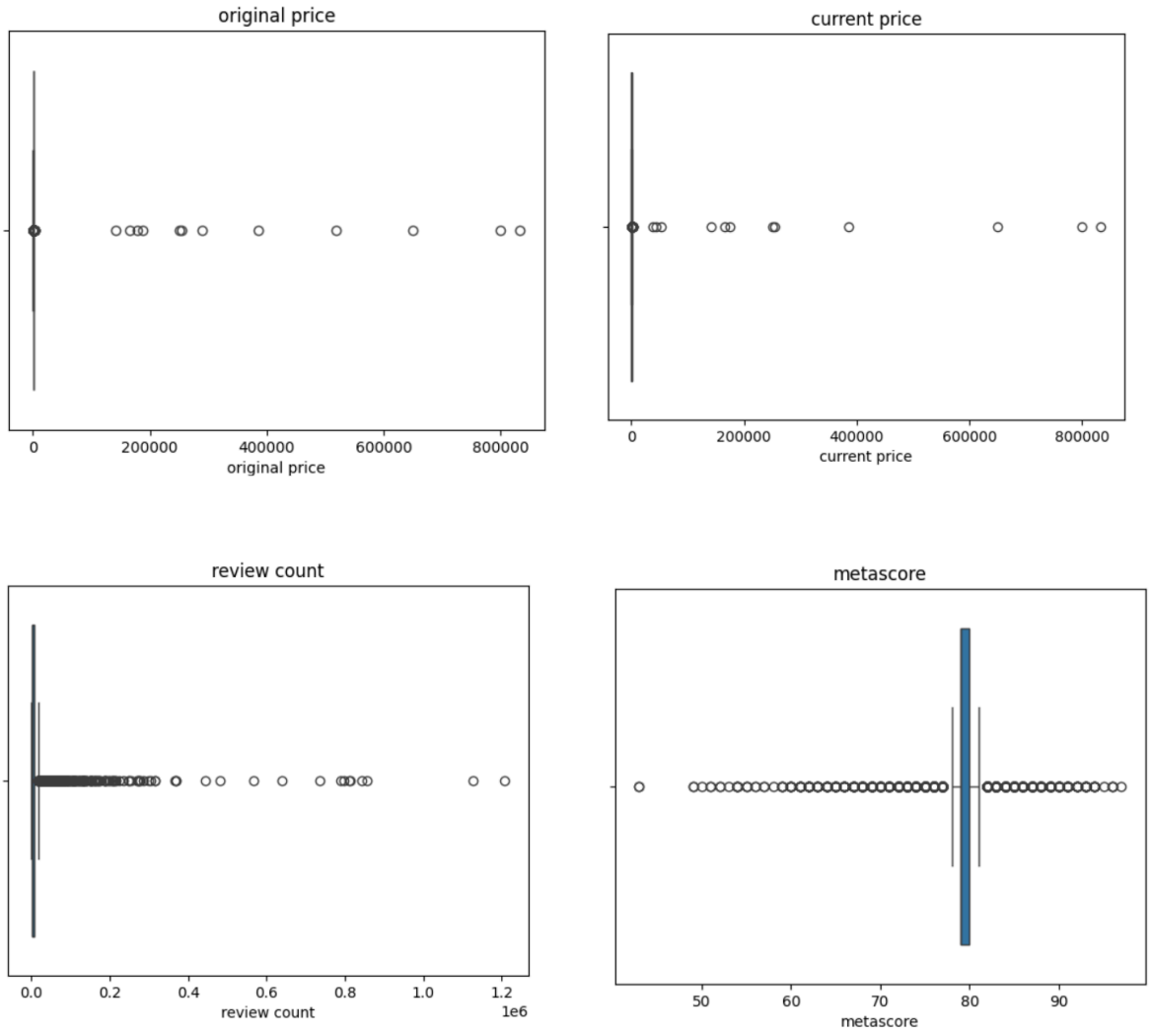


Figure: Outliers.

Why Outlier Removal Was Not Performed

Although outliers were identified through visualization, they were not removed for the following reasons:

- **Real-world validity**:
  Extreme values (e.g., very high prices or review counts) are genuine characteristics of the gaming market. Popular or premium games naturally have significantly higher values.
- **Preservation of important patterns**:
  Removing outliers could eliminate meaningful insights, especially when analyzing top-performing or high-revenue games.
- **Dataset context**:
  In domains like gaming, skewed distributions are expected rather than erroneous.

- **No evidence of data entry errors**:
  The outliers appeared to be legitimate observations rather than noise or mistakes.

Therefore, retaining outliers ensured that the dataset accurately reflects real-world market behavior.

9. Final Dataset Export

The cleaned dataset was saved for further analysis.

```
<class 'pandas.core.frame.DataFrame'>
Index: 3224 entries, 0 to 3277
Data columns (total 14 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   title           3224 non-null   object
 1   original price  3224 non-null   float64
 2   current price   3224 non-null   float64
 3   discount %      3224 non-null   int64
 4   review count    3224 non-null   int64
 5   tags            3224 non-null   object
 6   platform        3224 non-null   object
 7   release date    3224 non-null   datetime64[ns]
 8   metascore       3224 non-null   float64
 9   developer       3224 non-null   object
 10  publisher       3224 non-null   object
 11  summary         3224 non-null   object
 12  release year    3224 non-null   float64
 13  release month   3224 non-null   float64
```

```
title             0
original price    0
current price     0
discount %        0
review count      0
tags              0
platform          0
release date      0
metascore         0
developer         0
publisher         0
summary           0
release year      0
release month     0
dtype: int64
```

Figure: Post Preprocessing Dataset.

5.5 Machine Learning Models Used

This project employs a combination of traditional Machine Learning models and advanced Deep Learning techniques. Each model is applied to a specific task based on its suitability and strengths.

1. Popularity Classification Models

Used to classify games into High Popularity and Low Popularity categories.

- **Support Vector Machine (SVM)**
  Applied for binary classification using an RBF kernel to capture non-linear relationships between features such as price, metascore and release year.

- **Logistic Regression**
  Used as a baseline linear model for classification to compare performance with more complex models.
- **Random Forest Classifier**
  Used as an ensemble model to improve classification performance by combining multiple decision trees.

2. Discount Prediction Models

Used to predict whether a game will have a high discount (>50%).

- **Logistic Regression**
  Applied for binary classification after feature scaling and SMOTE balancing.
- **Decision Tree Classifier**
  Used to model non-linear relationships and to extract feature importance for interpretability.

3. Game Success Prediction Models (Regression)

Used to predict a continuous success score based on reviews and metascore.

- **Random Forest Regressor**
  Applied to model complex relationships and perform feature importance analysis.
- **XGBoost Regressor**
  Used as an advanced boosting algorithm with hyperparameter tuning to improve predictive performance over Random Forest.

4. Recommendation System Model

Used to recommend similar games based on content.

- **TF-IDF Vectorization + Cosine Similarity**
  Applied to convert textual features (tags and summaries) into numerical vectors and compute similarity between games.

5. Text Classification Models (Genre Prediction)

Used to classify game genres based on textual descriptions.

- **TF-IDF + Logistic Regression**
  Used as a traditional NLP baseline model for genre classification.

- **DistilBERT (Transformer Model)**
  Applied as a deep learning approach to capture contextual meaning in text and improve classification accuracy.

5.6 Independent and Dependent Variables

Independent Variables

Independent variables are the input features used by the Machine Learning models to predict game popularity, recommendation quality and other outcomes. These variables describe different characteristics of video games collected from the Steam dataset. The main independent variables used in this project are:

- **Current Price:**
  Represents the selling price of the game on the Steam platform. Price plays an important role in determining user interest and purchasing behavior.
- **Discount Percentage:**
  Indicates the amount of discount offered on a game. Games with larger discounts may attract more users and increase popularity.
- **Release Year and Game Age:**
  These variables help measure how recently a game was released. Newer games often receive more attention, while older games may maintain popularity due to reputation or fan base.
- **Genres and Tags:**
  Genres such as Action, Adventure, RPG, Casual and Indie, along with user-defined tags, help identify the type and style of games. These textual and categorical features are especially useful in recommendation systems and NLP tasks.
- **Metascore and User Ratings:**
  Critical reviews and player feedback help reflect overall game quality and strongly influence popularity prediction.
- **Summary:**
  Game descriptions, tags, and textual information were converted into numerical representations using TF-IDF and transformer tokenization methods for NLP-based experiments.

These independent variables were selected because they represent realistic factors that influence user preferences, game sales and overall success in the gaming industry.

Dependent Variables

Dependent variables are the outputs or target values that the models aim to predict. Different experiments in this project used different dependent variables depending on the task being performed.

- **Game Popularity Class (High or Low):**
  The primary dependent variable in the classification experiments was whether a game belonged to the "High Popularity" or "Low Popularity" category. Machine Learning models such as Logistic Regression, SVM, Random Forest and Decision Tree were trained to predict this category.
- **Recommendation Similarity Score:**
  In the recommendation system, the dependent output was the similarity score between games. Cosine similarity was used to measure how closely related one game is to another based on tags, genres and metadata.
- **Prediction Accuracy and Performance Metrics:**
  Model performance was evaluated using accuracy, precision, recall, F1-score, confusion matrices, MSE, MAE and R^2 score. These metrics were treated as important evaluation outputs for comparing different algorithms.
- **Text Classification Output:**
  For NLP experiments, the dependent variable was the predicted text classification label generated using TF-IDF + Logistic Regression and DistilBERT models.

The dependent variables were carefully selected to evaluate different aspects of gaming analytics, including popularity prediction, recommendation quality and NLP model effectiveness.

5.7 Training and Testing Method

To ensure reliable model evaluation, the dataset was divided into two separate subsets: a training set and a testing set. The training dataset was used to train the Machine Learning models, while the testing dataset was used to evaluate model performance on unseen data. This approach helps measure how well the trained models generalize to new examples instead of simply memorizing the training data.

A standard train-test split technique from Scikit-learn was used in the experiments. Approximately 80% of the dataset was allocated for training and 20% for testing using the following configuration:

```
X_train, X_test, y_train, y_test = train_test_split(
    features,
```

```
    target,
    Test_size = 0.2,
    Random_state = 42
)
```

After splitting the data, different Machine Learning models such as Logistic Regression, Support Vector Machine (SVM), Decision Tree, Random Forest and XGBoost were trained using the training dataset. The trained models then predicted outputs for the testing dataset.

5.8 Cross-Validation

Cross-validation was used to reduce overfitting and improve the robustness of the models. Instead of evaluating the model using only one train-test split, the training data was divided into multiple folds. The model was trained and validated several times using different subsets of the data.

In this project, 3-fold cross-validation (cv=3) was applied during hyperparameter tuning:

```
search = RandomizedSearchCV(
    rf,
    param_distributions = param_dist,
    n_iter = 20,
    cv = 3,
    scoring = 'r2',
    n_jobs = -1,
    verbose = 1
)
```

This means the training dataset was divided into three parts:

- Two parts were used for training,
- One part was used for validation,
- The process repeated three times using different validation folds.

The average performance across all folds was then used to select the best model.

5.9 Hyperparameter Tuning

Hyperparameter tuning was performed using RandomizedSearchCV to automatically search for the best combination of parameters for the Random Forest model.

The parameters tested included:

- Number of trees (n_estimators),
- Maximum tree depth (max_depth),
- Minimum samples required for splitting (min_samples_split),
- Minimum samples per leaf (min_samples_leaf),
- Maximum feature selection method (max_features).

Example parameter ranges used:

```
param_dist = {
    "n_estimators": [200, 300, 400],
    "max_depth": [10, 15, 20, 25],
    "min_samples_split": [2, 5, 10],
    "min_samples_leaf": [1, 2, 3],
    "max_features": ["sqrt", "log2"]
}
```

By tuning these parameters, the model achieved improved prediction accuracy and better generalization performance. Overall, the combination of train-test splitting, cross-validation and hyperparameter tuning helped create more reliable and accurate Machine Learning models for gaming analytics and popularity prediction tasks.

5.10 Evaluation Criteria

The performance of the Machine Learning models was evaluated using several standard evaluation metrics. Different metrics were selected depending on whether the task involved classification, regression, recommendation, or NLP-based prediction. These evaluation techniques helped measure the accuracy, reliability and overall effectiveness of the proposed models.

Accuracy

Accuracy measures the percentage of correctly predicted instances out of all predictions made by the model. It is one of the most commonly used metrics for classification tasks.

$$\text{Accuracy} = \frac{\{TP + TN\}}{TP + TN + FP + FN}$$

Where:

- TP = True Positives,
- TN = True Negatives,
- FP = False Positives,
- FN = False Negatives.

For example, the Support Vector Machine (SVM) and Random Forest models achieved approximately 71% accuracy in predicting whether a game belonged to the high-popularity or low-popularity category.

Precision

Precision measures how many of the predicted positive cases were actually correct. It is useful when false positive predictions need to be minimized.

$$\text{Precision} = \frac{TP}{TP + FP}$$

Higher precision indicates that the model makes fewer incorrect positive predictions.

Recall

Recall measures how many actual positive cases were correctly identified by the model. It evaluates the model's ability to detect relevant instances.

$$\text{Recall} = \frac{TP}{TP + FN}$$

A high recall score means the model successfully identifies most of the important positive cases.

F1-Score

The F1-score combines both precision and recall into a single metric using their harmonic mean. It is especially useful when the dataset is imbalanced.

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

In this project, F1-score was used to compare the balance between precision and recall across different classifiers.

Confusion Matrix

A confusion matrix provides a detailed summary of prediction results by showing:

- Correctly classified positive cases,
- Correctly classified negative cases,
- Incorrect classifications.

For example, the SVM model generated the following confusion matrix:

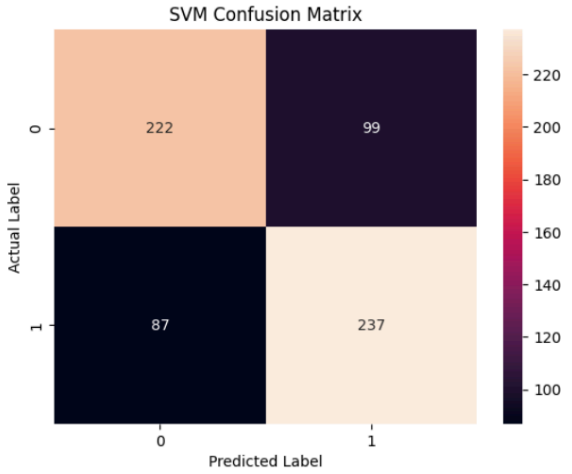


Figure: Confusion Matrix.

This indicates:

- 222 correctly predicted low-popular games,
- 237 correctly predicted high-popular games,
- 99 and 87 incorrect classifications.

Confusion matrices helped analyze model strengths and weaknesses more effectively than accuracy alone.

Mean Squared Error (MSE)

Mean Squared Error was used mainly for regression-based experiments such as Random Forest and XGBoost regression analysis.

$$\text{MSE} = \frac{1}{n} \Sigma_{i=1}^{n} \left(y_i - \widehat{y}_i\right) \left(y_i - \widehat{y}_i\right)$$

MSE measures the average squared difference between predicted and actual values. Lower MSE values indicate better prediction performance.

Mean Absolute Error (MAE)

MAE calculates the average absolute difference between predicted and actual values.

$$\text{MAE} = \frac{1}{n} \Sigma_{i=1}^{n} \left|y_i - \widehat{y_i}\right|$$

Unlike MSE, MAE does not square the errors, making it easier to interpret in practical situations.

R^2 Score

The R^2 score measures how well the regression model explains the variance in the target variable.

$$R^2 = 1 - \frac{\Sigma \left(y_i - \widehat{y}_i\right) \left(y_i - \widehat{y}_i\right)}{\Sigma \left(y_i - \widehat{y}\right) \left(y_i - \widehat{y}\right)}$$

An R^2 value closer to 1 indicates better model performance and stronger predictive capability.

In this project:

- Random Forest achieved an R^2 score of approximately 0.499,
- XGBoost achieved an R^2 score of approximately 0.53.

This showed that XGBoost slightly outperformed Random Forest in regression tasks.

5.11 Performance Analysis

The experiments showed that:

- Random Forest and SVM achieved strong classification performance with approximately 71% accuracy.
- Logistic Regression produced comparatively lower accuracy.
- XGBoost achieved good regression and feature importance results.
- DistilBERT significantly outperformed TF-IDF + Logistic Regression in text classification tasks.
- Feature importance analysis revealed that price, discounts, release year and Steam platform features strongly influence popularity prediction.

Comparison with Existing Methods

Unlike previous studies that focused only on popularity prediction or recommendation systems separately, this project combines:

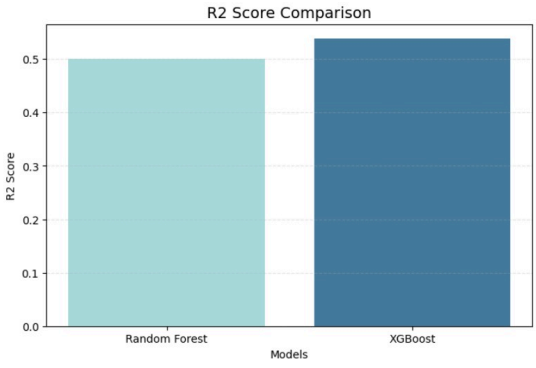
- Popularity classification,
- Recommendation systems,
- Feature importance analysis,
- NLP comparison,
- Transformer-based Deep Learning models.

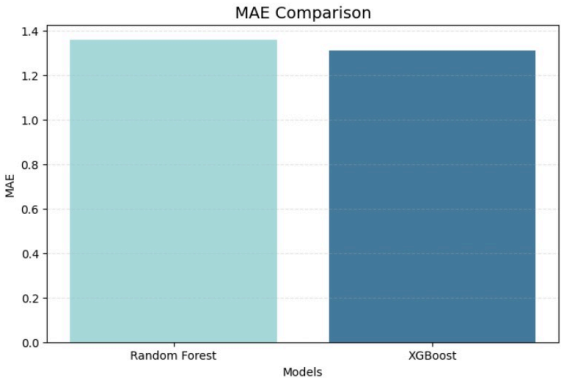
The combined approach provides a more complete gaming analytics framework and demonstrates how traditional Machine Learning and modern transformer architectures can work together for improved performance and deeper analysis.

6. Result and Analysis

6.1 Game Success Prediction

The performance comparison between the Random Forest and XGBoost regression models shows that XGBoost achieved better overall predictive accuracy for estimating game success. Based on the evaluation metrics, XGBoost outperformed Random Forest with a higher R^2 score and lower error values.





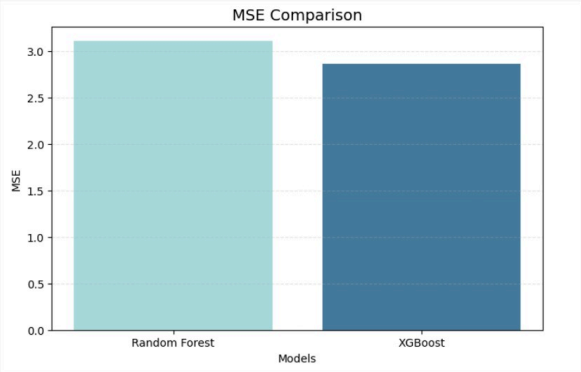


Figure: Comparison of Evaluation Score.

```
Fitting 3 folds for each of 20 candidates, totalling 60 fits
Random Forest Result
MSE: 3.106784828892005
MAE: 1.3642187647685802
R2 Score: 0.49992843367934414

Top 15 Important Features:
current price              0.105341
discount %                 0.104388
release year               0.091580
game_age                   0.084520
Steam Trading Cards        0.055939
Remote Play on Tablet      0.035111
Casual                     0.032450
release month              0.030991
Steam Achievements         0.023771
Remote Play on Phone       0.017298
Full controller support    0.016624
Multi-player               0.015986
Online Co-op               0.015145
Action                     0.013530
Steam Cloud                0.013023
dtype: float64
```

```
Fitting 3 folds for each of 20 candidates, totalling 60 fits
XGBoost Results
MSE: 2.920084665189548
MAE: 1.3131495911925035
R2 Score: 0.5299799011729289

Top 15 Important Features (XGBoost):
Steam Trading Cards                          0.074426
Remote Play on Tablet                        0.055196
developer_clean_Bitwave Games, Toaplan       0.037928
publisher_clean_BFG Entertainment            0.036413
Online Co-op                                 0.033437
discount %                                   0.027888
game_age                                     0.024381
Steam Achievements                           0.020105
publisher_clean_Kagura Games                 0.018466
Surround Sound                               0.017997
Casual                                       0.015797
publisher_clean_Shiravune                    0.015309
release year                                 0.015217
publisher_clean_CAPCOM Co., Ltd.             0.015044
Remote Play on Phone                         0.014884
dtype: float32
```

Figure: Top 15 Features of Random Forest and XGBoost Regression.

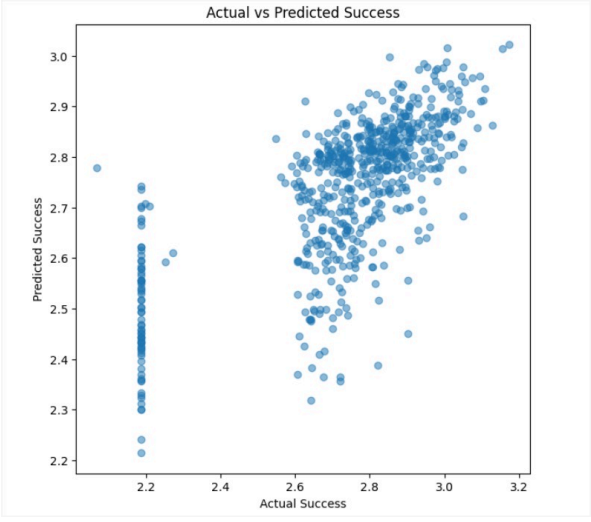


Figure: Actual VS Predicted Success

The scatter plot of actual versus predicted success values shows a positive correlation between the predicted and actual outputs. Most data points are concentrated around the central region, indicating that the model was able to capture the general trend of the dataset. However, some dispersion is still visible, suggesting that the models were not able to perfectly predict all success values.

6.2 Popular vs Non-popular Games

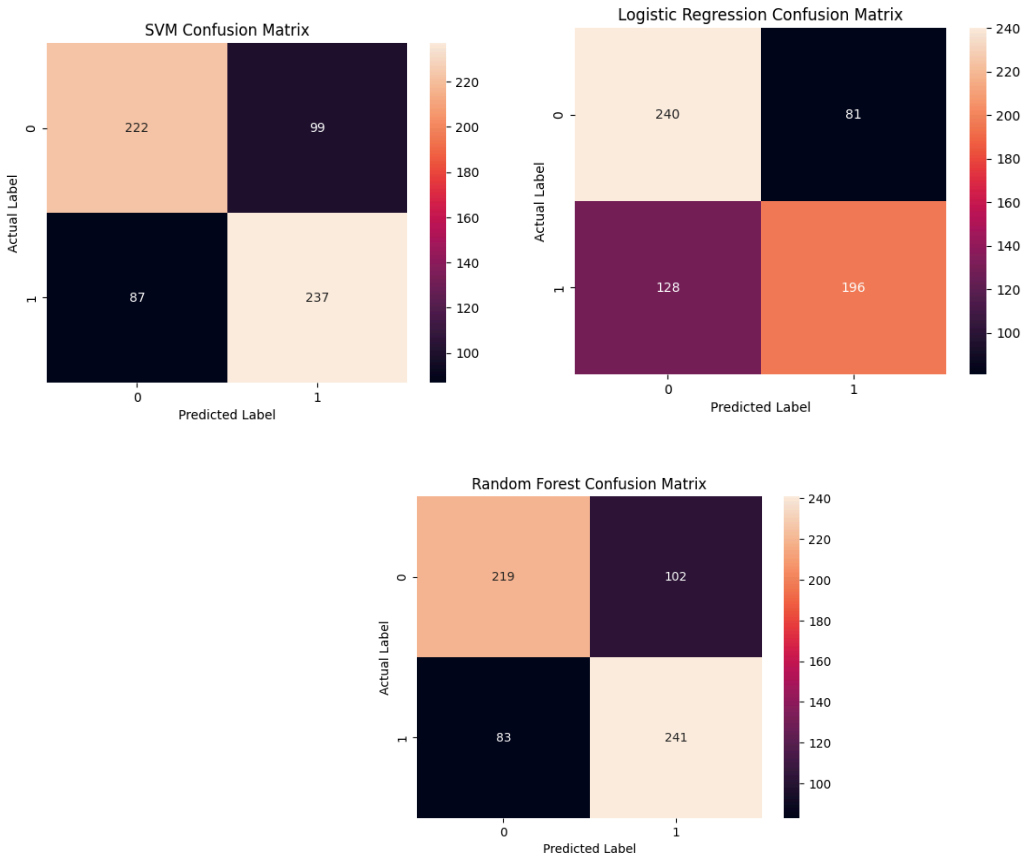


Figure: Confusion Matrix of Different Models.

```
--- Support Vector Machine ---

Classification Report:
              precision    recall  f1-score   support

           0       0.72      0.69      0.70       321
           1       0.71      0.73      0.72       324

    accuracy                           0.71       645
   macro avg       0.71      0.71      0.71       645
weighted avg       0.71      0.71      0.71       645
```

```
--- Logistic Regression ---

Classification Report:
              precision    recall  f1-score   support

           0       0.65      0.75      0.70       321
           1       0.71      0.60      0.65       324

    accuracy                           0.68       645
   macro avg       0.68      0.68      0.67       645
weighted avg       0.68      0.68      0.67       645
```

```
--- Random Forest ---

Classification Report:
              precision    recall  f1-score   support

           0       0.73      0.68      0.70       321
           1       0.70      0.74      0.72       324

    accuracy                           0.71       645
   macro avg       0.71      0.71      0.71       645
weighted avg       0.71      0.71      0.71       645
```

Figure: Evaluation Matrix.

The analysis shows that SVM and Random Forest achieved the highest accuracy of 71%, while Logistic Regression performed slightly lower at 68%.

SVM provided the most balanced performance with stable precision and recall for both classes. Random Forest performed similarly but was slightly better at detecting popular games (class 1), though it produced more false positives. Logistic Regression struggled more with identifying popular games, leading to lower recall and overall performance.

Overall, SVM and Random Forest were the most effective models for this classification task, with SVM giving the most balanced results.

6.3 High Discount Prediction

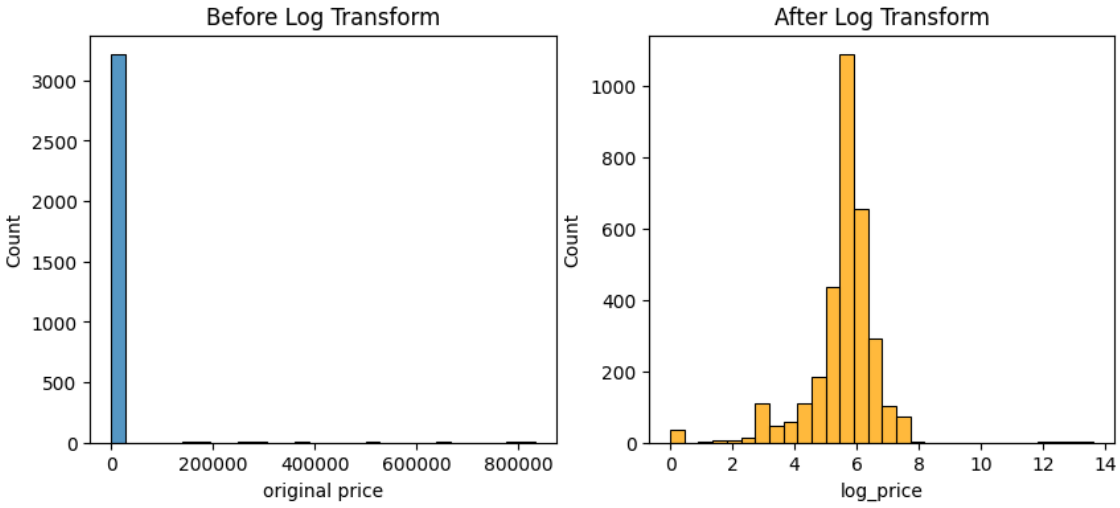


Figure: Visual Representation.

```
--- Logistic Regression Report ---
              precision    recall  f1-score   support

           0       0.78      0.64      0.70       478
           1       0.31      0.47      0.38       167

    accuracy                           0.60       645
   macro avg       0.55      0.56      0.54       645
weighted avg       0.66      0.60      0.62       645


--- Decision Tree Report ---
              precision    recall  f1-score   support

           0       0.90      0.64      0.75       478
           1       0.43      0.78      0.56       167

    accuracy                           0.68       645
   macro avg       0.66      0.71      0.65       645
weighted avg       0.78      0.68      0.70       645
```

	feature	importance
1	release year	0.417408
0	metascore	0.378527
32	log_price	0.096218
33	log_reviews	0.031291
11	Multi-player	0.029522
15	Strategy	0.015728
24	Steam Leaderboards	0.015429
27	Shared/Split Screen	0.005348
16	Co-op	0.004978
12	RPG	0.002138
14	Simulation	0.002138
5	Steam Cloud	0.000887
28	Remote Play on Tablet	0.000388
7	Full controller support	0.000000
9	Action	0.000000
8	Adventure	0.000000
3	Family Sharing	0.000000
2	Single-player	0.000000
17	Custom Volume Controls	0.000000
13	Casual	0.000000

Figure: Evaluation Matrix and Top Features.

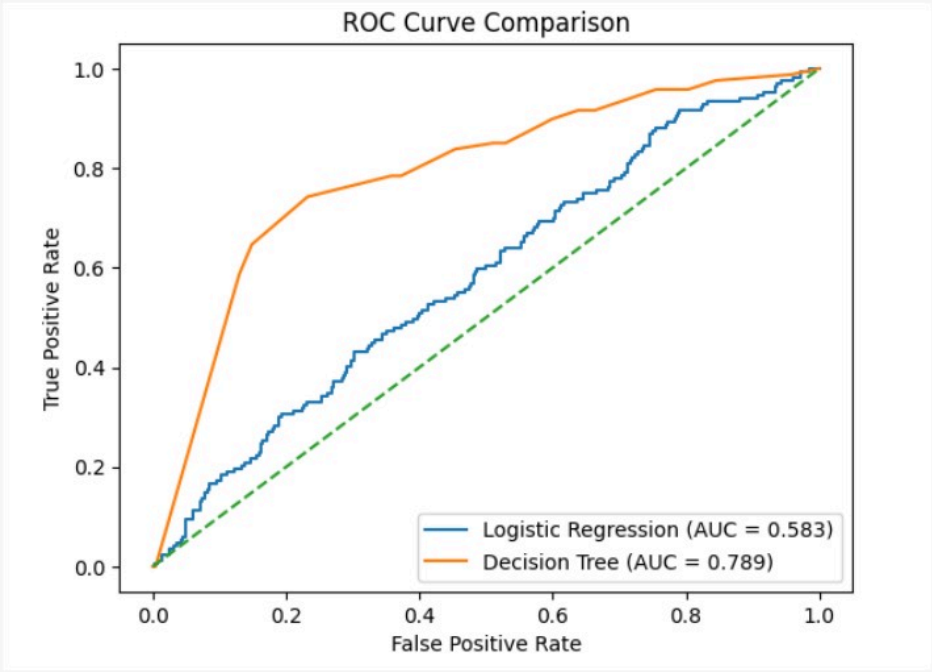


Figure: ROC - AUC Curve.

The comparative analysis shows that the Decision Tree model outperformed Logistic Regression in overall classification performance. Decision Tree achieved a higher accuracy of 68% and a stronger ROC-AUC score of 0.789, compared to 60% accuracy and 0.583 AUC for Logistic Regression. Decision Tree was especially better at identifying positive cases, with higher recall

and F1-score for class 1. The ROC curve also confirms that Decision Tree had better class separation ability.

Feature importance analysis revealed that release year, metascore, log price, and log reviews were the most influential factors in predicting game success. Additionally, the log transformation reduced skewness in price data, helping improve model performance.

6.4 Game Recommendation System

The recommendation system successfully identified games similar to the user's selected title, Resident Evil 2. The output suggests several related games such as Resident Evil 4 (2005) and Resident Evil 7 Biohazard, which share similar genres, gameplay styles and tags. Most recommended games contain tags like Action, Adventure and Single-player, indicating that the recommendation model effectively captures similarities based on game features and metadata.

The validation step also confirmed that Resident Evil 2 exists in the dataset, returning a value of True. This ensures that the recommendation system can correctly locate and process user-selected games before generating recommendations.

```
 you might like:
                          title  metascore  \
1082     Resident Evil 4 (2005)         79
1716                  Pinstripe         71
2918                Among Ashes         79
1035            Resident Evil 2         89
513   Resident Evil 7 Biohazard         83

                                                   tags
1082  Action, Adventure, Single-player, Steam Achiev...
1716  Adventure, Casual, Indie, Single-player, Steam...
2918  Action, Adventure, Indie, Single-player, Steam...
1035  Action, Single-player, Steam Achievements, Ful...
513   Action, Adventure, Single-player, Steam Achiev...
```

```
# Returns True or False
is_present = "Resident Evil 2" in df['title'].values
print(f"Is the game here? {is_present}")
```

```
Is the game here? True
```

Figure: Recommendation Output.

6.5 Game Genre Classification

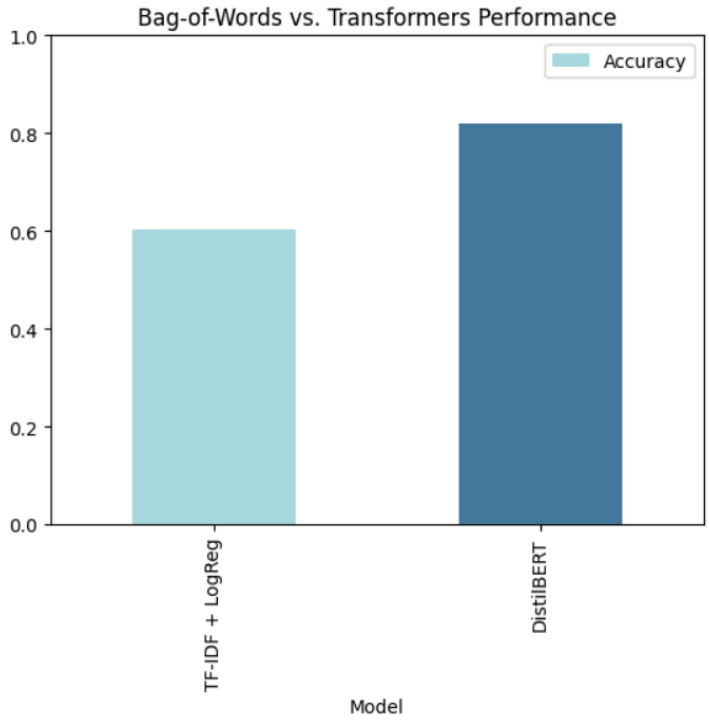


Figure: Comparison of TF-IDF and DistilBert.

It provides clear visual proof that modern Transformer models significantly outperform traditional statistical methods for video game genre classification.

The Performance Gap

- **Logistic Regression (LogReg):** The baseline model is hovering around 60% accuracy. In a 10-genre classification task, this is decent, but it means it's getting nearly half of its predictions wrong.
- **DistilBERT:** The transformer model jumped to approximately 82% accuracy. This 22% increase is a massive leap in the world of Natural Language Processing (NLP).

7. Discussion

The experimental results demonstrate that different Machine Learning and Deep Learning models have unique strengths for gaming analytics tasks. Ensemble learning methods such as Random Forest and XGBoost proved highly effective for prediction tasks because of their ability to capture complex relationships within the dataset. Support Vector Machine (SVM) provided balanced and reliable classification performance with stable precision and recall values across

different classes. Decision Tree models performed particularly well in handling non-linear pricing and discount patterns, making them suitable for discount prediction tasks. In the Natural Language Processing experiments, transformer-based models such as DistilBERT significantly outperformed traditional statistical approaches like TF-IDF with Logistic Regression, showing the advantage of contextual language understanding in genre classification. Furthermore, the content-based recommendation system successfully generated meaningful game suggestions by analyzing metadata, genres and tags. The results also emphasize the importance of proper data preprocessing, feature engineering and careful model selection in achieving strong predictive accuracy and reliable overall system performance.

8. Future Work

1. Integrate larger and more diverse datasets from platforms such as Epic Games, PlayStation and Xbox to improve model generalization and reduce platform-specific bias.

2. Incorporate additional user behavioral features such as playtime, wishlists, player retention and user interaction history for more accurate popularity prediction and personalized recommendations.

3. Implement advanced Deep Learning models such as Graph Neural Networks (GNNs), Recurrent Neural Networks (RNNs) and transformer-based architectures to improve recommendation and text analysis performance.

4. Develop hybrid and real-time recommendation systems by combining collaborative filtering with content-based filtering and using streaming data for adaptive recommendations.

5. Deploy the system as a web-based gaming analytics and recommendation platform to improve practical usability for both users and game developers.

9. Conclusion

This project demonstrates the successful application of Machine Learning and Deep Learning techniques for gaming analytics, popularity prediction, recommendation systems and text classification tasks using Steam game data. Multiple models including SVM, Logistic Regression, Decision Tree, Random Forest, XGBoost and DistilBERT were implemented and compared across different experiments. The results showed that ensemble learning methods such as Random Forest and XGBoost achieved strong predictive performance, while SVM provided balanced classification accuracy for popularity prediction tasks.

The study also confirmed that transformer-based models such as DistilBERT significantly outperform traditional NLP approaches like TF-IDF with Logistic Regression in genre classification tasks. In addition, the content-based recommendation system successfully generated meaningful game recommendations using tags, genres and metadata similarities. Feature importance analysis further revealed that factors such as price, discount percentage, release year, metascore and multiplayer-related features strongly influence game popularity and success.

Overall, the project highlights the importance of proper data preprocessing, feature engineering, model selection and evaluation in building effective gaming analytics systems. The combination of traditional Machine Learning methods with modern transformer-based Deep Learning models provides a comprehensive framework that can support recommendation systems, business decision-making and user experience improvement in modern gaming platforms.

10. Reference

1. https://arxiv.org/pdf/2110.02896

2. https://www.scitepress.org/Papers/2024/128538/128538.pdf

3. https://arxiv.org/pdf/2202.03392